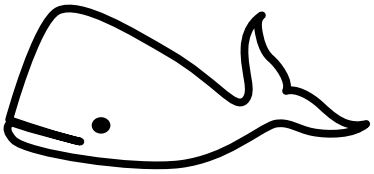
PEQUOD Manual

Parallel Quasi-Geostrophic Model

Version 0.9.5-dev



Dr. J. R. Maddison, Dr. P. S. Berloff,
Dr. S. A. Karabasov

November 8, 2026

Contents

1 Introduction **7**

2 Model equations **9**
2.1 Quasi-geostrophic potential vorticity equation 9
2.2 Background vertical shear 10
2.3 Boundary conditions 10
2.4 Layered quasi-geostrophic potential vorticity equation 12
2.5 Modal quasi-geostrophic potential vorticity equation 13
2.6 Forcing and dissipation 14

3 Numerical formulation **15**
3.1 Centred leapfrog scheme 15
3.1.1 Advection 15
3.1.2 Forcing and dissipation 16
3.1.3 Robert-Asselin-Williams time filter 17
3.1.4 Boundary conditions 17
3.1.5 Implementation details 18
3.2 CABARET scheme 18
3.2.1 Advection of relative potential vorticity 18
3.2.2 Advection of planetary vorticity 20
3.2.3 Forcing and dissipation 21
3.2.4 Implementation details 21
3.3 Potential vorticity inversion 22
3.3.1 Simply connected domain 22
3.3.2 Periodic channel 24

4 Parallelisation **27**
4.1 Quasi-geostrophic potential vorticity equation 27
4.2 Potential vorticity inversion 27
4.3 Domain decomposition 28

5 Compiling the model **31**
5.1 Dependencies 31
5.1.1 Required dependencies 31
5.1.2 Optional dependencies 32
5.2 Compilation 32
5.3 Unit tests 33

6 Model configuration **35**
6.1 config.h . . . 35
6.2 qg3p_params.F90 . . . 37
6.3 Supplementary parameter files . . . 45
6.3.1 Stratification parameters . . . 45
6.3.2 Shear parameters . . . 46
6.3.3 Checkpoint files . . . 46

7 Model output and diagnostics **47**
7.1 Model output . . . 47
7.1.1 Array indexing . . . 47
7.1.2 Non-dimensionalisation . . . 47
7.1.3 Output file format . . . 48
7.1.4 Output files . . . 52
7.2 Additional diagnostics . . . 56
7.2.1 Energy diagnostics . . . 57
7.2.2 Performance diagnostics . . . 57

8 Tools **59**
8.1 qg_decomp . . . 59
8.2 qg_pickup . . . 59
8.3 qg_merge_parallel . . . 60
8.4 qg_cleanup . . . 60

Symbols and notation

Symbol	Definition
∂_X	Partial differentiation with respect to a variable X.
D_t	Material, or advective, derivative.
$\underline{\nabla}_H$	Two dimensional horizontal del operator.
$J(X,Y)$	Determinant of the two dimensional Jacobian matrix formed by X and Y, $J(X,Y) = \partial_x X \partial_y Y - \partial_y X \partial_x Y$.
$\delta(\ldots)$	Dirac delta function.
$\underline{X}$	Vector.
X_i	ith element of a vector $\underline{X}$.
$\underline{\underline{X}}$	Matrix.
X_{ij}	i,jth element of a matrix $\underline{\underline{X}}$.
$\underline{\underline{I}}$	Identity matrix.
δ_{ij}	Kronecker delta, $I_{ij} = \delta_{ij}$.
x	Zonal coordinate.
y	Meridional coordinate.
z	Vertical coordinate.
q	Quasi-geostrophic potential vorticity (or "potential vorticity").
ζ	Relative quasi-geostrophic potential vorticity (or "relative potential vorticity").
ω	Relative vorticity.
ψ	Quasi-geostrophic stream function (or "stream function").
ρ	Fluid density.
ρ_0	Fluid reference density.
u	Zonal velocity (at zeroth order in Rossby number).
v	Meridional velocity (at zeroth order in Rossby number).
b	Buoyancy (at zeroth order in Rossby number).
w_a	Vertical velocity (at first order in Rossby number).
f	Planetary vorticity.

Symbol	Definition
f_0	Typical planetary vorticity at domain mid-y.
β	Meridional planetary vorticity gradient.
g	Gravitational acceleration.
g'	Reduced gravity.
ν	Coefficient of Laplacian viscosity for momentum.
r	Bottom drag coefficient.
τ	Wind stress per unit density.
F	Forcing and dissipation terms in the quasi-geostrophic potential vorticity equation.
$\underline{\underline{S}}$	Stratification matrix.
$\underline{\underline{D}}$	Diagonalisation matrix for $\underline{\underline{S}}$, such that $\underline{\underline{D}}\ \underline{\underline{S}}\ \underline{\underline{D}}^{-1} = \underline{\underline{\lambda}}$.
$\underline{\underline{\lambda}}$	Matrix of eigenvalues of $\underline{\underline{S}}$, $\lambda_{ij} = \delta_{ij} l_i$ where l_i is the ith eigenvalue for $\underline{\underline{S}}$.
$R_{D,i}$	Rossby radii of deformation.
$\underline{\underline{A}}_i$	Advection matrix for ith baroclinic mode.
$\underline{\tilde{X}}$	Baroclinic modal projection of a layer variable $\underline{X}$. $\underline{\tilde{X}} = \underline{\underline{D}}\ \underline{X}$.
Δx	Grid spacing.
Δt	Time step size.
N_i	Number of grid points in the zonal direction.
N_j	Number of grid points in the meridional direction.
$X_{i,j}$	Value of a quantity X at grid point i in the zonal direction and j in the meridional direction.
X^n	Value of a quantity X at time level n.
ν	Robert-Asselin filter parameter.
γ	Robert-Asselin-Williams (RAW) filter parameter.

Chapter 1

Introduction

PEQUOD (the Parallel Quasi-Geostrophic Model) is a finite difference code for solving the multi-layer quasi-geostrophic equations in a rectangular domain. The model has two primary modes of operation: basin mode and channel mode. Typically, the basin mode is used as a simple model for a wind driven double gyre, with energy input to the system via upper layer wind forcing. The channel mode is used as a simple model for baroclinic zonal jet formation, with the energy input to the system via lateral buoyancy forcing.

PEQUOD is primarily intended for the study of mesoscale eddy processes. To this end, PEQUOD implements the advanced CABARET scheme (the Compact Accurately Boundary Adjusting high-REsolution Technique), a form of second-order upwind scheme implementing a non-linear flux limiter. The model is fully parallelised using MPI. The combination of the high-resolution and stable CABARET scheme, together with MPI parallelism, makes PEQUOD suitable for high resolution eddy resolving numerical simulations in which multiple baroclinic deformation radii are fully resolved.

Chapter 2

Model equations

In this chapter the equations solved by PEQUOD are outlined. The quasi-geostrophic potential vorticity equation is introduced in section 2.1. The modification of this equation in the presence of a background vertical shear is discussed in section 2.2, and the boundary conditions stated in section 2.3. The level and layer formulations are described in section 2.4, and the projection onto vertical modes is described in section 2.5. Finally, forms of forcing and dissipation are stated in section 2.6.

2.1 Quasi-geostrophic potential vorticity equation

PEQUOD solves the quasi-geostrophic potential vorticity equation subject to forcing and dissipation. These equations arise from an asymptotic expansion of the momentum, mass continuity and thermodynamic equations, retaining terms up to first order in Rossby number. Only the key results are listed here. For a complete derivation, see Pedlosky [1987] and Vallis [2006].

At leading order in Rossby number we have geostrophic and hydrostatic balance:

$$u = -\partial_y \psi, \qquad v = \partial_x \psi, \qquad w = 0, \qquad b = f_0 \partial_z \psi, \tag{2.1}$$

where u, v, w, and b are the zeroth order contributions for the zonal velocity, meridional velocity, vertical velocity, and buoyancy respectively, and ψ is the quasi-geostrophic stream function (hereafter referred to as the "stream function"). At next order in Rossby number one obtains the quasi-geostrophic potential vorticity equation:

$$D_t q = F, \tag{2.2}$$

where q is the quasi-geostrophic potential vorticity (hereafter referred to as the "potential vorticity"), $D_t = \partial_t + J(\psi, q)$ is the material derivative, and F is any forcing and dissipation. The advection term is defined using the leading order horizontal velocity, and notated in terms of the determinant of the horizontal

Jacobian matrix formed by ψ and q:

$$\begin{aligned} J(\psi, q) &= \begin{vmatrix} \partial_x \psi & \partial_x q \\ \partial_y \psi & \partial_y q \end{vmatrix} \\ &= \partial_x \psi \partial_y q - \partial_y \psi \partial_x q \\ &= \underline{u} \cdot \underline{\nabla}_H q, \end{aligned} \tag{2.3}$$

where ∇_H is the horizontal del operator and $\underline{u} = (u, v)$. The potential vorticity is given by:

$$q = \nabla^2 \psi + \beta y + \partial_z \left(\frac{f_0^2}{N^2} \partial_z \psi \right), \tag{2.4}$$

where $f = f_0 + \beta y$ is the planetary vorticity subject to the β-plane approximation and N is the buoyancy frequency. The first term in equation (2.4) is the relative vorticity, the second term is the contribution due to the background vorticity gradient (arising from the planetary vorticity), and the third term is the contribution due to the vertical buoyancy gradient (the "thickness" term).

2.2 Background vertical shear

Let $0 \leq z \leq H$, and let the stream function be given by:

$$\psi = -\frac{Uyz}{H} + \psi'. \tag{2.5}$$

This is equivalent to a vertical shear in the x-direction, from a velocity of zero at the lower boundary to a velocity of U at the upper boundary, and with a deviation of ψ' from the background shear. The potential vorticity (2.4) becomes:

$$\begin{aligned} q &= \nabla^2 \psi' + \beta y + \partial_z \left(\frac{f_0^2}{N^2} \partial_z \psi' \right) - \frac{Uy}{H} \partial_z \left(\frac{f_0^2}{N^2} \right) \\ &= \nabla^2 \psi' + \tilde{\beta} y + \partial_z \left(\frac{f_0^2}{N^2} \partial_z \psi' \right), \end{aligned} \tag{2.6}$$

where $\tilde{\beta} = \beta - \frac{U}{H} \partial_z \left(\frac{f_0^2}{N^2} \right)$. The potential vorticity equation (2.2) becomes:

$$\partial_t q + J(\psi', q) + \frac{Uz}{H} \partial_x q = F. \tag{2.7}$$

Hence the presence of a background vertical shear is equivalent to a modification of the background vorticity gradient, combined with an additional linear advection of potential vorticity.

2.3 Boundary conditions

The quasi-geostrophic thermodynamic equation is:

$$D_t b + N^2 w_a = 0, \tag{2.8}$$

where w_a is the first order (in Rossby number) contribution for the vertical velocity (and the advection is by the leading order, geostrophic velocity). Applying free-slip boundary conditions at the upper and lower boundaries yields the boundary condition:

$$D_t b = f_0 D_t \partial_z \psi = 0 \quad \text{on} \quad \partial\Omega_U \cup \partial\Omega_B, \tag{2.9}$$

where $\partial\Omega_U$ and $\partial\Omega_B$ are the upper and lower boundaries respectively. Note that, in the presence of Ekman boundary layers, the right-hand size of (2.9) becomes non-zero.

Equivalently, following Bretherton [1966], the boundary condition (2.9) may be replaced by the homogeneous boundary condition :

$$\psi \to \psi^*, \quad \partial_z \psi^* = 0 \quad \text{on} \quad \partial\Omega_U \cup \partial\Omega_B, \tag{2.10}$$

provided the potential vorticity (2.4) is modified to:

$$\begin{aligned} q \to q^* &= \nabla^2 \psi^* + \beta y + \partial_z \left(\frac{f_0^2}{N^2} \partial_z \psi^* \right) \\ &= \nabla^2 \psi + \beta y + \partial_z \left(\frac{f_0^2}{N^2} \partial_z \psi \right) + q_B^*, \end{aligned} \tag{2.11}$$

where:

$$\int_{z-\delta z}^{z+\delta z} q_B^* dz' = \begin{cases} + \left(\frac{f_0}{N^2} b \right) \Big|_{H_B} & \text{if } H_B \in [z-\delta z, z+\delta z] \\ - \left(\frac{f_0}{N^2} b \right) \Big|_{H_U} & \text{if } H_U \in [z-\delta z, z+\delta z] \\ + \left(\frac{f_0}{N^2} b \right) \Big|_{H_B} - \left(\frac{f_0}{N^2} b \right) \Big|_{H_U} & \text{if } H_B, H_U \in [z-\delta z, z+\delta z] \\ 0 & \text{otherwise,} \end{cases} \tag{2.12}$$

where H_U and H_B are the z-coordinates of the upper and lower boundaries respectively. With some abuse of notation (2.11) can be expressed via:

$$\begin{aligned} q \to q^* = \nabla^2 \psi + \beta y + \partial_z \left(\frac{f_0^2}{N^2} \partial_z \psi \right) \\ + \lim_{\epsilon \to 0^+} \left[\frac{f_0}{N^2} b \left[\delta(z - H_B + \epsilon) - \delta(z - H_U - \epsilon) \right] \right]. \end{aligned} \tag{2.13}$$

Note that $b^* = f_0 \partial_z \psi^* = b = f_0 \partial_z \psi$ only in the open interval (H_B, H_U). See Schneider et al. [2003] for the origins of this boundary modified potential vorticity from the non-quasi-geostrophic case.

Restated, the potential vorticity equation (2.2) with a potential vorticity of the form (2.4), combined with the boundary condition (2.9), is (neglecting forcing and dissipation and Ekman boundary layers) a statement of material conservation of potential vorticity in the interior and material conservation of buoyancy on the upper and lower boundaries. It follows that this can be combined to a statement of material conservation of a single quantity, the boundary modified potential vorticity (2.11). This mathematical construction permits flow constraints and stability criteria to be formulated only in terms of the boundary modified potential vorticity.

2.4 Layered quasi-geostrophic potential vorticity equation

We consider a finite difference discretisation of equation (2.2) in the vertical dimension using centred finite differencing. Divide the vertical dimension into N levels and let the ith level have thickness h_i. Apply second order centred finite differencing in the vertical, subject to the boundary condition $\partial_z \psi = 0$ on the upper and lower boundaries. This choice of boundary condition is appropriate, provided we recognise that the potential vorticity has been substituted with the boundary modified potential vorticity as per equation (2.11). The boundary terms in (2.11) provide a contribution to the upper- and lowermost levels.

The equation for the potential vorticity (2.4) becomes:

$$\underline{\zeta} = \nabla^2 \underline{\psi} + \underline{\underline{S}}\, \underline{\psi}, \tag{2.14}$$

where $\zeta = q - \beta y$ is the relative potential vorticity, $\underline{\zeta}$ and $\underline{\psi}$ are vectors defining the relative potential vorticity and stream function in each level, and $\underline{\underline{S}}$ is a stratification matrix. The stratification matrix $\underline{\underline{S}}$ is defined such that, assuming $N \geq 2$:

$$\sum_{j=1}^{N} S_{ij}\psi_j = \begin{cases} s_1^+ (\psi_2 - \psi_1) & \text{if} \quad i = 1, \\ s_i^- (\psi_{i-1} - \psi_i) + s_i^+ (\psi_{i+1} - \psi_i) & \text{if} \quad 1 < i < N, \\ s_N^- (\psi_{N-1} - \psi_N) & \text{if} \quad i = N, \end{cases} \tag{2.15}$$

where:

$$\begin{aligned} s_i^- &= \frac{f_0^2}{N_{i-1/2}^2} \frac{2}{h_i (h_{i-1} + h_i)}, \\ s_i^+ &= \frac{f_0^2}{N_{i+1/2}^2} \frac{2}{h_i (h_{i+1} + h_i)}, \end{aligned} \tag{2.16}$$

and where $N_{i-1/2}$ is buoyancy frequency at the interface between the $(i-1)$ and ith levels, and $N_{i+1/2}$ is the buoyancy frequency at the interface between the i and $(i+1)$th levels. $i = 1$ and $i = N$, correspond to the upper- and lowermost levels respectively. The quasi-geostrophic potential vorticity equation (2.2) now becomes, for each level:

$$\partial_t q_i + J(\psi_i, q_i) = F_i. \tag{2.17}$$

The vertically discretised level model (2.17) with relative potential vorticity (2.14) is mathematically equivalent to a number of stacked quasi-geostrophic shallow-water layers [Vallis, 2006]. In such a layer model the interfacial buoyancy frequencies are replaced by:

$$\begin{aligned} N_{i-1/2}^2 &= \frac{2g'_{i-1/2}}{(h_{i-1} + h_i)}, \\ N_{i+1/2}^2 &= \frac{2g'_{i+1/2}}{(h_{i+1} + h_i)}, \end{aligned}$$

where $g'_{i-1/2}$ is the reduced gravity between the $i-1$ and ith layers, and $g'_{i+1/2}$ is the reduced gravity between the $i+1$ and ith layers, where:

$$g'_{i-1/2} = -g\frac{\rho_{i-1} - \rho_i}{\rho_0},$$
$$g'_{i+1/2} = -g\frac{\rho_i - \rho_{i+1}}{\rho_0}, \tag{2.18}$$

where g is the gravitational acceleration, ρ are the densities in each layer, and ρ_0 is a reference density.

Note that a second order finite volume discretisation in the vertical also yields the level formulation (2.17), making clear the equivalence between the level and layer formulations.

2.5 Modal quasi-geostrophic potential vorticity equation

Multiplying equation (2.14) by some matrix $\underline{\underline{D}}$ yields:

$$\underline{\underline{D}}\,\underline{\zeta} = \nabla^2\underline{\underline{D}}\,\underline{\psi} + \underline{\underline{D}}\,\underline{\underline{S}}\,\underline{\psi}. \tag{2.19}$$

We now choose $\underline{\underline{D}}$ such that:

$$\underline{\underline{D}}\,\underline{\underline{S}}\,\underline{\underline{D}}^{-1} = \underline{\underline{\lambda}}, \tag{2.20}$$

where $\underline{\underline{\lambda}}$ is a diagonal matrix. This defines an eigenvalue problem for $\underline{\underline{S}}$ [Strang, 1988], and it follows that $\underline{\underline{D}}^{-1}$ is a matrix whose columns are the eigenvectors of $\underline{\underline{S}}$ and the diagonal entries of $\underline{\underline{\lambda}}$ are the corresponding eigenvalues. Hence equation (2.19) becomes:

$$\underline{\underline{D}}\,\underline{\zeta} = \nabla^2\underline{\underline{D}}\,\underline{\psi} + \underline{\underline{\lambda}}\,\underline{\underline{D}}\,\underline{\psi}, \tag{2.21}$$

yielding a series of decoupled Helmholtz equations:

$$\tilde{\zeta}_i = \left(\nabla^2 + l_i\right)\tilde{\psi}_i, \tag{2.22}$$

where, for a variable X, $\tilde{X}_i = \sum_{j=1}^{N} D_{ij}X_j$, and where l_i is the ith eigenvalue of $\underline{\underline{S}}$.

Equation (2.20) is the vertically discretised eigenvalue problem for the vertical structure functions in a decomposition of the system in terms of vertical modes (compare with equation (6.12.7a) of Pedlosky [1987]). It follows that the corresponding eigenvalues l_i are defined in terms of the Rossby radii of deformation, $R_{D,i}$:

$$l_i = -\frac{1}{R_{D,i}^2}. \tag{2.23}$$

Hence $\tilde{X}_i$ denotes X for the ith baroclinic mode. Note that provided the deformation radii are distinct, it follows that $\underline{\underline{D}}^{-1}$ is invertible [Strang, 1988].

Hereafter, the barotropic mode (with $R_D = \infty$) is associated with $i = 1$, and the remaining modes (the baroclinic modes) with $i \geq 2$.

The quasi-geostrophic potential vorticity equation (2.2) now becomes, for each mode:

$$\partial_t \tilde{q}_i + \sum_{i=j}^{N} \sum_{j=k}^{N} A_{ijk} J\left(\tilde{\psi}_j, \tilde{q}_k\right) = \tilde{F}_i, \tag{2.24}$$

where:

$$A_{ijk} = \sum_{l=1}^{N} D_{il} D_{lj}^{-1} D_{lk}^{-1}. \tag{2.25}$$

2.6 Forcing and dissipation

A wind forcing applied to the uppermost layer takes the form:

$$F_{W,i} = \delta_{i1} \frac{\left(\nabla_H \times \tau\right)_z}{h_1}, \tag{2.26}$$

where $\left(\nabla_H \times \tau\right)_z$ is the vertical component of the wind stress curl per unit density, and τ is the wind stress per unit density.

We consider two forms of dissipation:

$$F_{D,i} = \nu \nabla^4 \psi_i - r \delta_{iN} \nabla^2 \psi_i. \tag{2.27}$$

The fourth order term is equivalent to Laplacian viscosity for momentum, and acts to dissipate small scale enstrophy, q^2, from the system. The second order term, applied only to the bottom-most layer, represents the presence of an Ekman layer. ν is a viscosity coefficient and r is a bottom drag coefficient.

Note that the fourth order term results in an additional boundary condition requirement. The system can be closed via a Dirichlet boundary condition for $\nabla^2 \omega$, where $\omega = \nabla^2 \psi$ is the relative vorticity. One such boundary condition is partial-slip which, for a boundary aligned parallel to the y-axis and at $x = x_L$, takes the form:

$$\omega\left(x = x_L\right) = \left.\partial_{xx} \psi\right|_{x=x_L} = \alpha \left.\partial_x \psi\right|_{x=x_L}, \tag{2.28}$$

where $\omega = \nabla^2 \psi$ is the relative vorticity and α is a partial-slip parameter (an inverse length scale associated with the parameterised boundary layer). $\alpha \to \infty$ yields a no-slip boundary condition, while $\alpha = 0$ yields a free-slip boundary condition.

Chapter 3

Numerical formulation

PEQUOD implements two distinct numerical discretisations of the layered quasi-geostrophic potential vorticity equation (2.17). The first scheme uses a centred-in-space and centred-in-time discretisation. The second scheme utilises the Compact Accurately Boundary Adjusting high-REsolution Technique (CABARET), a flux limited second order upwind scheme.

Section 3.1 describes the centred scheme, while section 3.2 describes the CABARET scheme. In these two sections it is assumed that the stream function is available. In section 3.3 the method by which the potential vorticity is inverted for the stream function is described.

3.1 Centred leapfrog scheme

This section describes the centred leapfrog scheme. Section 3.1.1 details the advection scheme, section 3.1.2 details the forcing and dissipation, section 3.1.3 describes the time filter used to damp the computational mode associated with centred differencing in time, and section 3.1.4 describes the boundary conditions. Finally, section 3.1.5 describes some details associated with the implementation of this scheme by the model.

In this section $(\ldots)_{i,j}$ denotes grid coordinate i in the zonal direction and j in the meridional direction. Unless otherwise stated, all equations are defined for $2 \leq i \leq (N_i - 1)$ and $2 \leq j \leq (N_j - 1)$, where N_i and N_j are the number of grid points in the zonal and meridional directions respectively. $(\ldots)^n$ denotes time level n. The time step size and grid spacing are given by Δt and Δx respectively.

3.1.1 Advection

In the centred leapfrog scheme the advection term is cast to flux form:

$$J(\psi, q) = \nabla \cdot (\underline{u} q), \tag{3.1}$$

where this identity holds as $\nabla_H \cdot \underline{u} = 0$. The operator is discretised using centred differencing in space and time to yield the advection tendency:

$$\begin{aligned}
\frac{q^{n+1}_{i,j} - q^{n-1}_{i,j}}{2\Delta t} = & \frac{1}{2\Delta x}\left[u^n_{i-1,j} q^n_{i-1,j} - u^n_{i+1,j} q^n_{i+1,j}\right] \\
+ & \frac{1}{2\Delta x}\left[v^n_{i,j-1} q^n_{i,j-1} - v^n_{i,j+1} q^n_{i,j+1}\right] \\
= & \frac{1}{4\Delta x^2}\left[-\left(\psi^n_{i-1,j+1} - \psi^n_{i-1,j-1}\right) q^n_{i-1,j}\right. \\
& \qquad \left. + \left(\psi^n_{i+1,j+1} - \psi^n_{i+1,j-1}\right) q^n_{i+1,j}\right] \\
+ & \frac{1}{4\Delta x^2}\left[\left(\psi^n_{i+1,j-1} - \psi^n_{i-1,j-1}\right) q^n_{i,j-1}\right. \\
& \qquad \left. - \left(\psi^n_{i+1,j+1} - \psi^n_{i-1,j+1}\right) q^n_{i,j+1}\right]. \qquad (3.2)
\end{aligned}$$

This corresponds to the discrete Jacobian $\mathbb{J}^{+\times}(q, \psi)$ in Arakawa [1966]. In the model terms corresponding to $J(\psi_i, \psi_i)$ are discarded for efficiency.

Since this is a discretisation of the potential vorticity advective flux, potential vorticity is conserved. This method is second order accurate in space and time.

3.1.2 Forcing and dissipation

A time independent forcing, such as a time independent wind stress applied to the upper layer, is discretised using centred-in-time differencing. This leads to a tendency of:

$$\frac{q^{n+1}_{i,j} - q^{n-1}_{i,j}}{2\Delta t} = F_W. \qquad (3.3)$$

A stable discretisation of the tendency resulting from fourth order diffusion, described in section 2.6, is centred differencing in space and a simple forward Euler discretisation applied between time levels $n-1$ and $n+1$:

$$\frac{q^{n+1}_{i,j} - q^{n-1}_{i,j}}{2\Delta t} = \frac{\nu}{\Delta x^2}\left[\omega^{n-1}_{i,j+1} + \omega^{n-1}_{i-1,j} - 4\omega^{n-1}_{i,j} + \omega^{n-1}_{i+1,j} + \omega^{n-1}_{i,j-1}\right]. \qquad (3.4)$$

The relative vorticity is computed using centred finite differencing:

$$\omega^n_{i,j} = \frac{1}{\Delta x^2}\left[\psi^n_{i,j+1} + \psi^n_{i-1,j} - 4\psi^n_{i,j} + \psi^n_{i+1,j} + \psi^n_{i,j-1}\right], \qquad (3.5)$$

where the value of ω on the boundaries is determined as described in section 3.1.4. The resulting discretisation of the fourth order diffusion term is second order accurate in space and first order accurate in time.

Similarly, the tendency due to the second order bottom drag, as discussed in section 2.6, is discretised using a forward Euler discretisation between time levels $n-1$ and $n+1$:

$$\frac{q^{n+1}_{i,j} - q^{n-1}_{i,j}}{2\Delta t} = -r\omega^{n-1}_{i,j}, \qquad (3.6)$$

with second order accuracy in space and first order accuracy in time.

Note that a centred discretisation in space and time is unstable for diffusion [Haltiner and Williams, 1980].

3.1.3 Robert-Asselin-Williams time filter

The use of centred differencing in time is second order accurate, but a computational mode exists which can grow over time and pollute the resulting solution. This mode can be damped via a time filter. The time filter used is a modified Robert-Asselin time filter as proposed by Williams [2009] – the Robert-Asselin-Williams (RAW) filter. Centred differencing in time for a tendency $\mathcal{F}^n\left(q^n\right)$ is modified to:

$$\frac{q^{n+1} - \overline{\overline{q}}^{n-1}}{2\Delta t} = \mathcal{F}^n\left(\overline{q}^n\right), \tag{3.7}$$

where:

$$\begin{aligned} \overline{q}^{n+1} &= q^{n+1} - \frac{\mu\left(1-\gamma\right)}{2}\left[\overline{\overline{q}}^{n-1} - 2\overline{q}^n + q^{n+1}\right], \\ \overline{\overline{q}}^{n} &= \overline{q}^n + \frac{\mu\gamma}{2}\left[\overline{\overline{q}}^{n-1} - 2\overline{q}^n + q^{n+1}\right], \end{aligned} \tag{3.8}$$

where $0 \leq \mu, \gamma \leq 1$. μ is a parameter controlling the strength of the filter. For $\gamma = 1$ equation (3.8) reduces to the standard Robert-Asselin filter [Robert, 1966, Asselin, 1972], and the discretisation is unconditionally stable and first order accurate in time. For $\gamma = 1/2$ the discretisation is third order accurate in time and unconditionally unstable. For $\gamma = 1/2 + \epsilon$ where $\epsilon > 0$ and $\epsilon \ll 1$ the discretisation is quasi-third order accurate in time (although formally first order accurate) and conditionally stable. For a full description of the modified filter, including a derivation of the stability and accuracy properties, see Williams [2009].

3.1.4 Boundary conditions

The fourth order diffusion term in section 3.1.2 requires an additional boundary condition for the relative vorticity ω. An appropriate boundary condition is a partial-slip Dirichlet boundary condition (see section 2.6).

Consider a boundary aligned parallel to the y-axis. Letting $i = 1$ on the boundary, a Taylor expansion retaining terms up to third order yields:

$$\psi_2 = \psi_1 + \partial_x\psi|_{x=x_L}\,\Delta x + \frac{1}{2}\,\partial_{xx}\psi|_{x=x_L}\,\Delta x^2 + \frac{1}{6}\,\partial_{xxx}\psi|_{x=x_L}\,\Delta x^3, \tag{3.9a}$$

$$\psi_3 = \psi_1 + 2\,\partial_x\psi|_{x=x_L}\,\Delta x + 2\,\partial_{xx}\psi|_{x=x_L}\,\Delta x^2 + \frac{4}{3}\,\partial_{xxx}\psi|_{x=x_L}\,\Delta x^3. \tag{3.9b}$$

Eliminating the third order terms and applying a partial-slip boundary condition:

$$\omega_1 = \partial_{xx}\psi|_{x=x_L} = \alpha\,\partial_x\psi|_{x=x_L}, \tag{3.10}$$

yields:

$$\omega_1 = \partial_{xx}\psi|_{x=x_L} = \frac{-7\psi_1 + 8\psi_2 - \psi_3}{2\Delta x^2 + \frac{6}{\alpha}\Delta x}. \tag{3.11}$$

Alternatively, a no-slip boundary condition can be defined via:

$$\omega_1 = \partial_{xx}\psi|_{x=x_L} = \frac{2\psi_2 - 2\psi_1}{\Delta x^2}, \tag{3.12}$$

which is equivalent to asserting that the tangential velocity on the boundary, averaged between $(x_L + \Delta x/2)$ and an image point at $(x_L - \Delta x/2)$, is zero.

Note that the boundary is defined to be co-incident with end nodes. Hence corner cells have one quarter the area of interior cells, and other boundary cells have one half the area of interior cells.

3.1.5 Implementation details

1. The relative potential vorticity ζ is used as the fundamental prognostic model variable.

2. The advection term in the layer form of the quasi-geostrophic potential vorticity equation (2.17) yields terms proportional to $J(\psi_i, \psi_i)$. These terms are discarded for efficiency.

3. The values of the relative potential vorticity on the boundaries are computed from the stream function and relative vorticity, following application of the boundary condition for the relative vorticity.

4. On the first timestep a single forward Euler step of size Δt is used, prior to leapfrogging in time.

3.2 CABARET scheme

The second numerical method implemented in PEQUOD incorporates the Compact Accurately Boundary Adjusting high-REsolution Technique (CABARET) for advection of relative potential vorticity, combined with integration of the advection of planetary vorticity and the forcing and dissipation. This section provides a summary of the model implementation of the CABARET scheme, and is based on Karabasov et al. [2009].

In this section $(\ldots)_{i,j}$ denotes grid coordinate i in the zonal direction and j in the meridional direction. $(\ldots)_{i+1/2,j}$ denotes a point on a grid staggered in the zonal direction, at a point between grid coordinate i and $i+1$. $(\ldots)_{i,j+1/2}$ denotes a point on a grid staggered in the meridional direction, at a point between grid coordinate j and $j+1$. $(\ldots)_{i+1/2,j+1/2}$ denotes a point on a grid staggered in both the zonal and meridional directions. Unless otherwise stated all equations are defined excluding points lying on the boundary. $(\ldots)^n$ denote time level n, and $(\ldots)^{n+1/2}$ denote a time level intermediate between time levels n and $n+1$. The time step size and grid spacing are given by Δt and Δx respectively.

3.2.1 Advection of relative potential vorticity

The CABARET scheme is a second-order (in space and time) accurate method incorporating a non-linear flux limiter. This section provides a simple overview of the numerical method. Further details can be found in Karabasov and Goloviznin [2007] and Karabasov et al. [2009].

We consider the CABARET scheme in one dimension. The linear advection of a quantity q takes the form:

$$\partial_t q + \partial_x (uq) = 0, \tag{3.13}$$

or re-written:

$$\partial_t q + \partial_x f = 0, \tag{3.14}$$

where $f = uq$ is the advective flux. Let q and f be represented on a staggered grid with spacing Δx. Grid points for q are denoted using integer values for the grid coordinate, and grid points for f are denoted using half-integer values.

The CABARET scheme consists of three steps:

1. Predictor step: Integration of q^n at time level t to form $q^{n+1/2}$ at time level $t + 1/2\Delta t$ using a forward Euler discretisation.
2. Extrapolator step: Calculation of f^{n+1} from $q^{n+1/2}$ and f^n.
3. Corrector step: Integration from $q^{n+1/2}$ at time level $t + 1/2\Delta t$ to form q^{n+1} at time level $t + \Delta t$ using a backward Euler discretisation, using the fluxes computed at the extrapolator step.

The predictor and extrapolator step are performed using centred differencing in space. The predictor step is given by:

$$\frac{q_i^{n+1/2} - q_i^n}{1/2\Delta t} = -\frac{f_{i+1/2}^n - f_{i-1/2}^n}{\Delta x}, \tag{3.15}$$

and the corrector step by:

$$\frac{q_i^{n+1} - q_i^{n+1/2}}{1/2\Delta t} = -\frac{f_{i+1/2}^{n+1} - f_{i-1/2}^{n+1}}{\Delta x}, \tag{3.16}$$

The extrapolator step is performed using upwinding of q [Karabasov et al., 2009]. This takes the form:

$$q_{i+1/2}^{n+1} = 2q_i^{n+1/2} - q_{i-1/2}^n \quad \text{if} \quad u \geq 0, \tag{3.17a}$$

$$q_{i-1/2}^{n+1} = 2q_i^{n+1/2} - q_{i+1/2}^n \quad \text{if} \quad u < 0, \tag{3.17b}$$

from which the fluxes $f_{i-1/2}^{n+1} = uq_{i-1/2}^{n+1}$ can be computed.

This scheme is second order accurate provided the method used to compute the fluxes is (at least) second order accurate [Karabasov et al., 2009]. This can be achieved via simple interpolation of q onto the flux grid points via:

$$q_{i+1/2} = \frac{q_{i+1} + q_i}{2}. \tag{3.18}$$

The scheme as presented is a second order linear method, and as such is prone to the development of spurious overshoots. A flux limiter is introduced into the CABARET scheme by applying a bounding in the extrapolator step [Karabasov et al., 2009]:

$$q_{i+1/2}^{n+1} \to \max\left(\min\left(q_{i+1/2}^{n+1}, q_i^{n+1/2}, q_{i-1/2}^n\right), q_i^{n+1/2}, q_{i-1/2}^n\right) \quad \text{if} \quad u \geq 0, \tag{3.19a}$$

$$q_{i-1/2}^{n+1} \to \max\left(\min\left(q_{i-1/2}^{n+1}, q_i^{n+1/2}, q_{i+1/2}^n\right), q_i^{n+1/2}, q_{i+1/2}^n\right) \quad \text{if} \quad u < 0, \tag{3.19b}$$

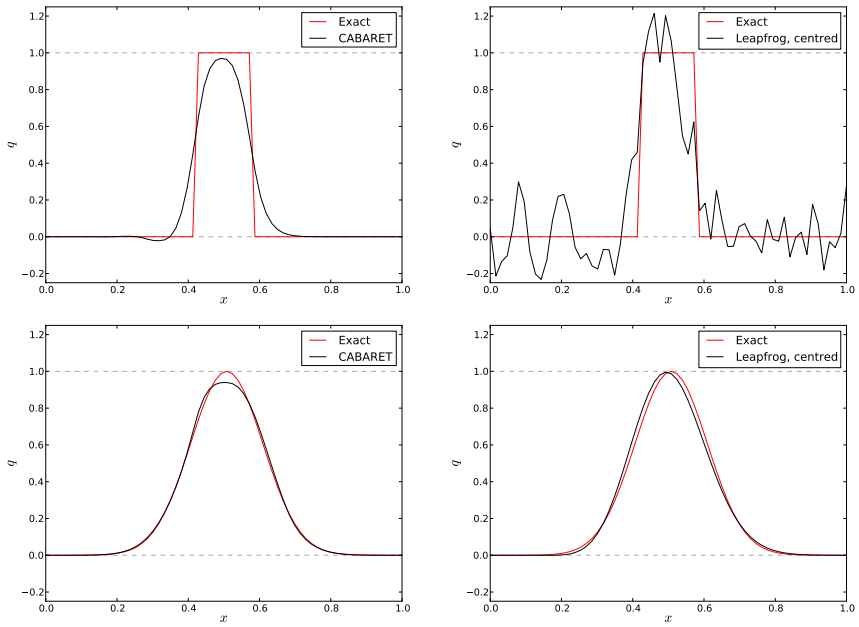


Figure 3.1: One-dimensional advection by a constant velocity at Courant number 0.1 for one circuit of a periodic domain consisting of 64 uniformly spaced grid points. Above: A top hat of width 10 grid points. Below: A Gaussian with standard deviation 0.1 times the domain size. Left: CABARET scheme. Right: Centred-in-space, centred-in-time (leapfrog) scheme (with no time filter).

This flux limiter is not TVD (Total Variation Diminishing) [Karabasov et al., 2009], and hence spurious maxima can still be generated by the CABARET scheme. A simple one-dimensional comparison of CABARET with a centred leapfrog discretisation is shown in figure 3.1.

In PEQUOD the CABARET scheme is applied only for the advection of the relative potential vorticity. See [Karabasov et al., 2009] for full details of the implementation. Since CABARET is a discretisation of the flux form advection tendency, the application to the advection of relative potential vorticity ensures conservation of the relative potential vorticity. CABARET is stable in one dimension for a Courant number ≤ 1, and in two dimensions for a Courant number ≤ 0.5.

3.2.2 Advection of planetary vorticity

The advection of planetary vorticity is treated using a second order scheme of the form:

$$\frac{q^{n+1/2}_{i,j} - q^{n-1/2}_{i,j}}{\Delta t} = -\frac{3}{2}\left[\frac{1}{2}\left(v^{n}_{i,j+1/2} + v^{n}_{i,j-1/2}\right)\beta\right] - \frac{1}{2}\left[\frac{1}{2}\left(v^{n-1}_{i,j+1/2} + v^{n-1}_{i,j-1/2}\right)\beta\right]. \tag{3.20}$$

See Karabasov et al. [2009] for further details.

This has second order accuracy in space and time. Note that this conserves the planetary vorticity (which follows as the velocity is divergence free). Hence,

since CABARET conserves the relative potential vorticity, the total potential vorticity is also conserved [Karabasov et al., 2009].

3.2.3 Forcing and dissipation

A time independent forcing, such as a time independent wind stress applied to the upper layer, is discretised using forward Euler. This leads to a tendency of:

$$\frac{q_{i,j}^{n+1/2} - q_{i,j}^{n-1/2}}{\Delta t} = F_W. \tag{3.21}$$

Diffusion is discretised using a backward scheme in time. This becomes, for the fourth order diffusion tendency (see section 2.6):

$$\frac{q_{i,j}^{n+1/2} - q_{i,j}^{n-1/2}}{\Delta t} = \frac{\nu}{\Delta x^2}\left[\omega_{i,j+1}^{n+1/2} + \omega_{i-1,j}^{n+1/2} - 4\omega_{i,j}^{n+1/2} + \omega_{i+1,j}^{n+1/2} + \omega_{i,j-1}^{n+1/2}\right]. \tag{3.22}$$

The relative vorticity is computed using centred finite differencing:

$$\omega_{i,j}^{n+1/2} = \frac{1}{\Delta x^2}\left[\psi_{i,j+1}^{n+1/2} + \psi_{i-1,j}^{n+1/2} - 4\psi_{i,j}^{n+1/2} + \psi_{i+1,j}^{n+1/2} + \psi_{i,j-1}^{n+1/2}\right], \tag{3.23}$$

where the value of ω on the boundaries is determined as described in section 3.1.4. The resulting discretisation of the fourth order diffusion term is second order accurate in space.

Similarly, the tendency due to the second order bottom drag (see section 2.6) is discretised via:

$$\frac{q_{i,j}^{n+1/2} - q_{i,j}^{n-1/2}}{\Delta t} = -r\omega_{i,j}^{n+1/2}, \tag{3.24}$$

with second order accuracy in space.

3.2.4 Implementation details

1. The relative potential vorticity ζ is used as the fundamental prognostic model variable.
2. The values of the relative potential vorticity on the boundaries are computed from the stream function and relative vorticity, following application of the boundary condition for the relative vorticity.
3. On the first timestep the terms associated with time level $n < 0$ are assigned a value of zero.
4. The potential vorticity and stream function are collocated in time at time levels $n+1/2$. Hence model diagnostics are computed at time levels $n+1/2$.
5. The CABARET implementation supports a time varying time step size. Following calculation of u^n a new time step size, $\Delta t^{n+1/2}$, is chosen such that:
$$\frac{||u^n||_\infty \Delta t^{n+1/2}}{\Delta x} = C,$$
where C is a prescribed target Courant number. See Karabasov et al. [2009] for a description of the modification of the time discretisation to support a varying time step size.

3.3 Potential vorticity inversion

This section describes the inversion of the potential vorticity q to yield the stream function ψ. In terms of vertical modes the inversion problem is as per equation (2.22), i.e. for the ith mode:

$$\left(\nabla^2 + l_i\right)\tilde{\psi}_i = \tilde{\zeta}_i = \tilde{q}_i - \beta y. \tag{3.25}$$

This equation can be discretised in space using centred differencing for the Laplace operator to yield:

$$\tilde{\psi}_{i,j+1,k} + \tilde{\psi}_{i-1,j,k} + (l_k - 4)\,\tilde{\psi}_{i,j,k} + \tilde{\psi}_{i+1,j,k} + \tilde{\psi}_{i,j-1,k} = \tilde{\zeta}_{i,j,k}, \tag{3.26}$$

where $X_{i,j,k}$ is the value of X at node i in the zonal direction, j in the meridional direction and for mode k, and $2 \leq i \leq (N_i - 1)$, $2 \leq j \leq (N_j - 1)$. This describes an implicit elliptic problem. Dirichlet boundary conditions are required to close the system, and the boundary condition terms contribute to the right-hand-side of the linear problem. The linear system can be written:

$$\sum_{l=1}^{N_i}\sum_{m=1}^{N_j} L_{lmijk}\tilde{\psi}_{lmk} = \tilde{\zeta}_{ijk} - \tilde{g}_{ijk}, \tag{3.27}$$

or, more compactly:

$$L_i\tilde{\psi}_i = \tilde{\zeta}_i - \tilde{g}_i, \tag{3.28}$$

where L_i is the discrete Helmholtz matrix and $-\tilde{g}_i$ are the Dirichlet boundary condition contributions to the right-hand-side for each mode.

Given Dirichlet boundary conditions, the resulting linear elliptic problem is inverted using a direct solver. A discrete sine transform is performed in the meridional direction, yielding a series of sparse one-dimensional Helmholtz problems in the zonal direction that can, for example, be inverted via Gaussian elimination. An inverse discrete sine transform of the resulting solutions of the one-dimensional problems completes the inversion. See Hogg et al. [2006] for a complete description of the method.

By the no-normal-flow boundary condition for $\underline{u}$ it follows that the $\tilde{\psi}_i$ are equal to constants on each boundary:

$$\tilde{\psi}_i = c_{ij} \quad \text{on} \quad \partial\Omega_j, \tag{3.29}$$

for each connected sub-surface $\partial\Omega_j$ of the domain Ω. As stated the c_{ij} are not uniquely determined, and hence further constraints are required in order to yield a unique set of $\tilde{\psi}_i$ and close the linear problem [McWilliams, 1977].

Section 3.3.1 details the inversion problem in a simply connected domain, and section 3.3.2 details the inversion problem in a periodic channel.

3.3.1 Simply connected domain

For a simply connected domain there is a single boundary constant per mode. The value of the constant for the barotropic mode ($l_i = 0$) is arbitrary [McWilliams, 1977] and can be set to any constant.

An appropriate constraint for a simply connected domain is conservation of mass. In the continuous case this implies that, at each z:

$$\int_\Omega w_a = 0. \tag{3.30}$$

By equation (2.9) this leads to, at each z:

$$\partial_t \int_\Omega \psi = 0. \tag{3.31}$$

Assuming that $\int_\Omega \psi = 0$ at some time (e.g. initially) this leads to, in the layer model case:

$$\int_\Omega \tilde{\psi}_i = 0, \tag{3.32}$$

for all modes excluding the barotropic mode. For consistency, since the boundary constant is arbitrary for the barotropic mode, this constraint can also be applied for this mode.

Define $\tilde{\psi}_i^0$ and $\tilde{\psi}_i^1$ such that:

$$\begin{aligned} L_i\tilde{\psi}_i^0 &= \tilde{\zeta}_i, & \tilde{\psi}^0 &= 0 \quad \text{on} \quad \partial\Omega, \\ L_i\tilde{\psi}_i^1 &= 1, & \tilde{\psi}^1 &= 0 \quad \text{on} \quad \partial\Omega, \end{aligned} \tag{3.33}$$

Then:

$$L_i\left(\tilde{\psi}_i^0 + k_i\tilde{\psi}_i^1\right) = \tilde{\zeta}_i + k_i, \tag{3.34}$$

which leads to:

$$L_i\left[\tilde{\psi}_i^0 + k_i\left(\tilde{\psi}_i^1 - \frac{1}{l_i}\right)\right] = \tilde{\zeta}_i. \tag{3.35}$$

Hence we can write:

$$\tilde{\psi}_i = \tilde{\psi}_i^0 + k_i\left(\tilde{\psi}_i^1 - \frac{1}{l_i}\right), \tag{3.36}$$

and, since $\tilde{\psi}^0 = \tilde{\psi}^1 = 0$ on $\partial\Omega$:

$$c_i = -\frac{k_i}{l_i}. \tag{3.37}$$

By the integral constraint (3.32):

$$\begin{aligned} \int_\Omega \tilde{\psi}_i = 0 \Rightarrow \int_\Omega \tilde{\psi}_i^0 + k_i\left(\int_\Omega \tilde{\psi}_i^1 - \frac{1}{l_i}\int_\Omega\right) &= 0 \\ \Rightarrow k_i &= -\frac{\int_\Omega \tilde{\psi}_i^0}{\int_\Omega \tilde{\psi}_i^1 - \frac{1}{l_i}\int_\Omega} \\ \Rightarrow k_i &= \frac{l_i\overline{\tilde{\psi}_i^0}}{1 - l_i\overline{\tilde{\psi}_i^1}} \end{aligned} \tag{3.38}$$

where $\overline{(\ldots)}$ denotes "domain-averaged". Hence:

$$\tilde{\psi}_i = \tilde{\psi}_i^0 - \overline{\tilde{\psi}_i^0}\left(\frac{1 - l_i\tilde{\psi}_i^1}{1 - l_i\overline{\tilde{\psi}_i^1}}\right), \tag{3.39}$$

Hence the potential vorticity inversion consists of two linear solves for $\tilde{\psi}_i^0$ and $\tilde{\psi}_i^1$, as per equation (3.33), together with the calculation of $\tilde{\psi}_i$ as per equation (3.39). The solution for the $\tilde{\psi}_i^1$ are performed once per simulation, and cached.

3.3.2 Periodic channel

The mass conservation constraint is insufficient for the multiply connected case. Additional constraints can be derived by imposing momentum conservation constraints [McWilliams, 1977]. For details of the numerical implementation of such constraints, see Hogg et al. [2006].

In PEQUOD, when using a periodic channel geometry, the following lateral boundary conditions are used for the barotropic mode:

$$\tilde{\psi}_1(x,0) = \tilde{\psi}_1(x,L_y), \tag{3.40}$$

where the lateral boundaries are located at $y = 0$ and $y = L_y$. This boundary condition imposes zero net barotropic shear. Since only one constraint is required for the barotropic mode in this geometry [McWilliams, 1977], one can arbitrarily set:

$$\tilde{\psi}_1(x,0) = \tilde{\psi}_1(x,L_y) = 0. \tag{3.41}$$

The following boundary conditions are used for the baroclinic modes:

$$\int_{x=0}^{L_x} \tilde{\zeta}_i dx \Big|_{y=0,L_y} = 0 \qquad i \geq 2, \tag{3.42}$$

applied for the $(i-1)$th baroclinic mode, and where the domain has size L_x in the x-dimension. The discrete relative potential vorticity on the boundary can be written as a function of the discrete stream function, via the partial-slip boundary condition (3.11) or no-slip boundary condition (3.12), and hence this yields constraints for the $\tilde{\psi}_i$ $(i \geq 2)$.

Note that this choice of boundary conditions does not yield momentum or mass conservation. However, when combined with a background vertical shear as discussed in section 2.2, the baroclinic boundary conditions (3.42) ensure that the vertical gradient of the background, x-averaged, net horizontal potential vorticity gradient across the domain is not eroded by the dynamics. Equivalently, the baroclinic boundary conditions can be considered a form of lateral buoyancy forcing, which acts to maintain the vertical gradient of the background, x-averaged, net horizontal potential vorticity gradient across the domain.

These boundary conditions are applied in the potential vorticity inversion in the following manner. Define $\tilde{\psi}_i^0$, $\tilde{\psi}_i^1$, and $\tilde{\psi}_i^2$, such that:

$$\tilde{\psi}_i = \tilde{\psi}_i^0 + c_{i1}\tilde{\psi}_i^1 + c_{i2}\tilde{\psi}_i^2, \tag{3.43}$$

where:

$$\begin{array}{lllllllll}
L_i\tilde{\psi}_i^0 = \tilde{\zeta}_i, & \tilde{\psi}^0 = 0 & \text{on} & \partial\Omega_S, & \tilde{\psi}^0 = 0 & \text{on} & \partial\Omega_N, \\
L_i\tilde{\psi}_i^1 = 0, & \tilde{\psi}^1 = 0 & \text{on} & \partial\Omega_S, & \tilde{\psi}^1 = 1 & \text{on} & \partial\Omega_N, \\
L_i\tilde{\psi}_i^2 = 0, & \tilde{\psi}^2 = 1 & \text{on} & \partial\Omega_S, & \tilde{\psi}^2 = 0 & \text{on} & \partial\Omega_N,
\end{array} \tag{3.44}$$

and where $\partial\Omega_S$ and $\partial\Omega_N$ are the southern and northern boundaries respectively.

For the barotropic mode $i = 0$ and, by (3.41), $c_{11} = c_{12} = 0$. Define:

$$I_1\left(\tilde{\psi}_i\right) = \int_{x=0}^{L_x} \tilde{\zeta}_i dx \Big|_{y=0},$$
$$I_2\left(\tilde{\psi}_i\right) = \int_{x=0}^{L_x} \tilde{\zeta}_i dx \Big|_{y=L_y}, \tag{3.45}$$

where the integrals on the right-hand-side are evaluated from the $\tilde{\psi}_i$ via the partial-slip boundary condition (3.11) or the no-slip boundary condition (3.12). Then, since I_1 and I_2 are linear, and using the boundary condition (3.42), for each baroclinic mode ($i \geq 2$):

$$c_{i1} I_1\left(\tilde{\psi}_i^1\right) + c_{i2} I_1\left(\tilde{\psi}_i^2\right) = -I_1\left(\tilde{\psi}_i^0\right),$$
$$c_{i1} I_2\left(\tilde{\psi}_i^1\right) + c_{i2} I_2\left(\tilde{\psi}_i^2\right) = -I_2\left(\tilde{\psi}_i^0\right). \tag{3.46}$$

This forms a 2×2 linear system for the c_{ij}, $i \geq 2$.

Hence the potential vorticity inversion for the barotropic mode consists of one linear solve for $\tilde{\psi}_1 = \tilde{\psi}_1^0$. The potential vorticity inversion for each baroclinic mode consists of three linear solves for $\tilde{\psi}_i^0$, $\tilde{\psi}_i^1$, and $\tilde{\psi}_i^2$, as per equation (3.43), together with the calculation of the c_{ij} via an inversion of a 2×2 linear system for the c_{ij}. The solutions for the $\tilde{\psi}_i^1$, and $\tilde{\psi}_i^2$ are performed once per simulation, and cached.

Chapter 4

Parallelisation

This section provides a brief overview of the parallelisation method implemented within PEQUOD.

The model is parallelised for distributed memory machines using MPI . A given simulation is distributed across a given number of processes by designating a sub-set of the model domain to each process. This defines the partitioning of the problem, with each process assigned a partition of the global domain.

Section 4.1 describes the parallelisation of the explicit timestepping of the quasi-geostrophic potential vorticity equation (2.2). Section 4.2 describes the parallelisation of the implicit potential vorticity inversion step, defined by equation (2.4). Details concerning the domain decomposition are described in section 4.3.

4.1 Quasi-geostrophic potential vorticity equation

The numerical solution of the quasi-geostrophic potential vorticity equation (2.2), using either the centred scheme or CABARET, involves the solution of purely explicit problems – no matrix inversion operations are required (all left-hand-side matrices are the identity matrix). However, differencing at partition boundaries requires information to be communicated from adjacent processes.

This is achieved via the use of halo (or ghost) nodes. The local partition assigned to each process is extended into neighbouring partitions, generating an overlap region known as the "halo". The size of the halo (the number of nodes in the overlap) is dependent upon the stencil of the discrete operator. Information must be communicated periodically from processes "owning" the nodes in the halo region (processes assigned the halo nodes in the original partitioning) to all other processes which must access those nodes in differencing.

4.2 Potential vorticity inversion

The potential vorticity inversion step, defined by equation (2.4) is an implicit problem – the Helmholtz problems must be inverted in order to obtain the stream function from the relative potential vorticity. Furthermore, the ellipticity

of the Helmholtz operator means that this is inherently a non-local procedure – information must be communicated from all nodes to all other nodes, and hence from all processes to all other processes, in order to perform the inversion.

This step is parallelised using a combination of data transposing and a parallel solver for the one-dimensional Helmholtz problems. The system is repartitioned (via communication) yielding a new partitioning that is contiguous in the meridional direction (data transposing). This allows the discrete sine transform in the meridional direction to be performed locally on each process. The one-dimensional Helmholtz problems are inverted in parallel using the Schumann and Strietzel [1995] algorithm. The inverse discrete sine transform step can be performed locally. Finally, the system is repartitioned back to the original partitioning.

4.3 Domain decomposition

The potential vorticity inversion step involves a great deal of communication. Hence it is advisable to choose a model partitioning that avoids the need for repartitioning via data transposing. This is achieved by partitioning the model only in the longitudinal direction.

Figure 4.1 shows the relative parallel speedup of PEQUOD on the HECToR phase 3 for a three-layer wind driven square basin. These scaling tests use a decomposition only in the longitudinal direction, and use the parallel tridiagonal solver of Schumann and Strietzel [1995]. The tests are performed using PEQUOD version 0.9.3 and with PEQUOD compiled using the Cray Fortran compiler version 8.0.1. Nodes on the HECToR phase 3 have 32 cores, and the tests shown use only fully populated nodes. The use of a single processor for the 513×513 grid case has been found to be significantly more efficient than the use of a fully populated node. This is possibly due to reduced data contention, and could explain the super-linear speedup observed for the CABARET integration with the 513×513 and 1025×1025 grids.

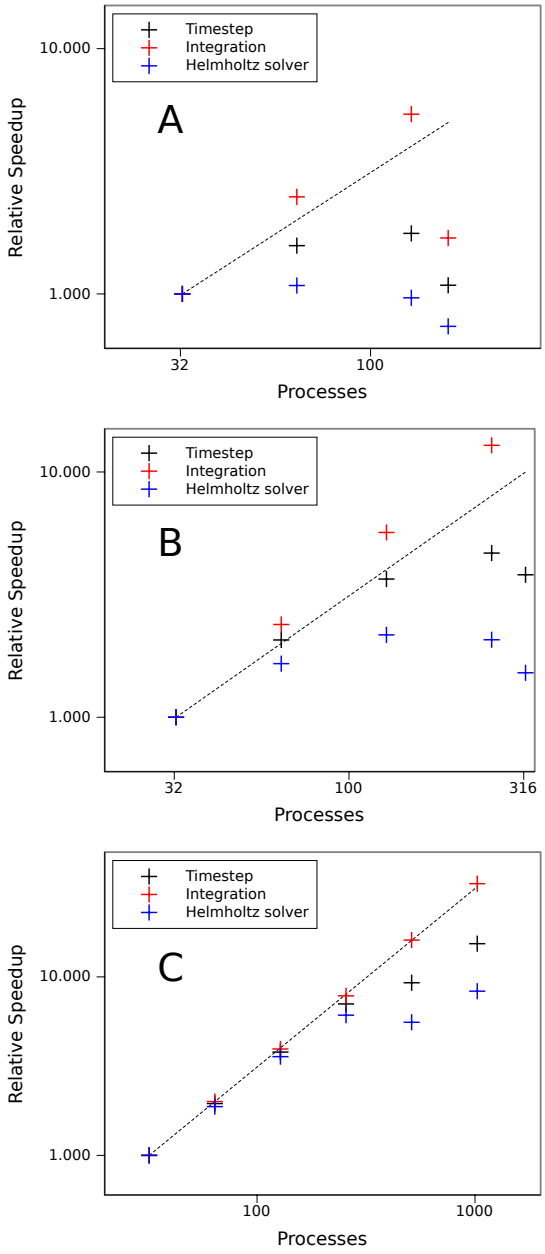


Figure 4.1: Relative parallel speedup of PEQUOD running with the CABARET scheme in a three-layer wind-driven square basin. All timings are recorded on the HECToR phase 3 using the performance diagnostics described in section 7.2.2. "Timestep" refers to the total timestep runtime, "Integration" refers to the total time spent integrating the quasi-geostrophic potential vorticity equation, and "Helmholtz" refers to the total time spent performing the potential vorticity inversion. All timings exclude initialisation times. A: 513×513 grid. B: 1025×1025 grid. C: 4097×4097 grid.

Chapter 5

Compiling the model

This section describes how to compile PEQUOD. Section 5.1 describes model dependencies, section 5.2 describes compiler options, and section 5.3 describes model unit tests. Note that the model must be recompiled each time the configuration files `config.h` or `qg3p_params.F90` are modified (see section 6).

5.1 Dependencies

A small number of tools and libraries are required to compile PEQUOD. These are typically available as part of a standard system, and hence you should not normally need to compile them yourself. However, links to free and open source versions are provided.

5.1.1 Required dependencies

The following are required for compilation of PEQUOD.

Make

A standard tool used to assist in code compilation. GNU Make can be downloaded from `http://www.gnu.org/software/make/`.

Fortran compiler

PEQUOD in written primarily in Fortran 95, and makes use of a small number of Fortran 2003 features. As such a fairly modern Fortran compiler is required. PEQUOD has been compiled successfully with GCC version 4.6.3, and with the Cray Fortran compiler version 8.0.4. GCC can be downloaded from `http://gcc.gnu.org/`.

MPI

An MPI implementation is required to compile and run the model. PEQUOD has been tested with Open MPI versions 1.4.3 and 1.6.1. Open MPI can be downloaded from `http://www.open-mpi.org/`.

LAPACK

LAPACK is a library of fundamental linear algebra routines. An implementation of LAPACK is available from `http://www.netlib.org/lapack/`. Note that optimised versions are usually available on clusters.

5.1.2 Optional dependencies

The following are optional. They may be used by model scripts or to build documentation, or may be required when using experimental model features.

Python

Python is used by several model scripts, including scripts used by the model unit testing framework (see section 5.3). PEQUOD scripts have been used successfully with Python version 2.7.3. Python can be downloaded from `http://python.org/`.

NumPy

Some model scripts make use of the Python NumPy module. NumPy can be downloaded from `http://numpy.scipy.org/`. PEQUOD scripts have been used successfully with NumPy version 1.6.1.

LaTeX

This manual is written in LaTeX. Tex Live can be downloaded from `http://www.tug.org/texlive/`.

PETSc

PETSc is a library used to assist in the creation of numerical models, and is required to compile PEQUOD when using the experimental `PETSC` option in `config.h` (see section 6.1). PEQUOD has been compiled successfully with PETSc version 3.1. PETSc can be downloaded from `http://www.mcs.anl.gov/petsc/`.

5.2 Compilation

PEQUOD is compiled via make. The file `makefile.compiler` can be used to switch between compilers. For example:

```
include makefile.gfortran
```

selects the GNU fortran compiler, while:

```
include makefile.hector.cray
```

selects the Cray compiler for use on HECToR. Each compiler makefile defines a series of variables that select the compiler and compiler flags. In particular, the user may wish to modify:

Macro	Description
INCLUDES	Options defining directories containing include files.
LIBS	Options for linking against external libraries.
DEBUG	Debugging build compiler flags.
OPTIMISATION	Optimised build compiler flags.

In addition, the variable FCFLAGS defines options for the compilation of free form Fortran files. While this should not generally be modified, each compiler makefile contains two definitions for the variable, one with debugging enabled and one with optimisation enabled. Commenting out the appropriate version enables or disables debugging and optimisation. For example, in makefile.gfortran, debugging is enabled via:

```
#FCFLAGS = $(OPTIMISATION) $(FLAGS) \
  -ffree-line-length-none -DNDEBUG $(DEFINES) $(INCLUDES)
FCFLAGS = $(DEBUG) $(FLAGS) -ffree-line-length-none \
  -DDDEBUG $(DEFINES) $(INCLUDES)
```

while optimisation is enabled via:

```
FCFLAGS = $(OPTIMISATION) $(FLAGS) \
  -ffree-line-length-none -DNDEBUG $(DEFINES) $(INCLUDES)
#FCFLAGS = $(DEBUG) $(FLAGS) -ffree-line-length-none \
  -DDDEBUG $(DEFINES) $(INCLUDES)
```

Once a compiler makefile is selected and configured, the model is compiled via:

```
make qg3
```

Note that some compiler makefiles require the definition of the environment variable MPI_DIR. This should be set to the path of the local MPI installation (typically /usr/).

5.3 Unit tests

PEQUOD contains a number of small test programs. These test only a very small subset of model features. Hence the passing of these tests is a necessary, but certainly not sufficient, requirement for correct model functionality. The exact tests run depends upon the configuration as defined in config.h (see section 6.1).

The unit tests are compiled and run with the command:

```
make unittest
```

Note that the environment variable `MPI_DIR` must be defined in order to execute the unit tests. This environment variable should be set to the path of the local MPI installation (typically `/usr/`).

Chapter 6

Model configuration

This section describes the configuration of PEQUOD. The model is primarily configured via compile-time options, defined in two files. `config.h` contains preprocessor definitions that enable and disable features within the model. Options within this file are described in section 6.1. `qg3p_params.F90` contains all other compile-time model parameters, and is described in section 6.2. Finally, section 6.3 describes additional run-time parameter files that may be specified.

The model must be recompiled each time either `config.h` or `qg3p_params.F90` are modified (see section 5.2).

6.1 config.h

`config.h` contains preprocessor definitions that enable and disable model features. Options are enabled by defining a preprocessor macro, for example:

```
#define WIND_FORCING
```

enables the option `WIND_FORCING`, while:

```
#undef WIND_FORCING
```

disables the option.

The following options enable and disable terms in the quasi-geostrophic potential vorticity equation.

Macro	Description
`WIND_FORCING`	Upper layer wind forcing.
`BACKGROUND_VSHEAR`	Background vertical shear.
`BACKGROUND_HSHEAR`	Background horizontal shear.

The following options enable and disable CABARET features.

Macro	Description
CABARET	Enables the CABARET scheme. If disabled, the centred scheme is used.
CABARET_LIMITER	Enables the CABARET limiter.

The following options enable and disable features for the centred scheme. These are primarily legacy options.

Macro	Description
RA_DOUBLE_FILTER	Enables the Robert-Asselin-Williams time filter. If disabled, the standard Robert-Asselin filter is used. Recommended enabled.
CENTRED_BOTTOM_FRICTION	If enabled, bottom friction is discretised using a centred scheme in time, rather than a forward scheme. Recommended disabled.
FORWARD_EULER_START	If enabled, a forward Euler timestep is used for the first timestep. Otherwise the first timestep is initialised in an identical manner to the initial condition (i.e. zero valued or random valued relative potential vorticity). Recommended enabled.

The following options control the elliptic solver parallelisation. These are advanced or experimental options that are used primarily for testing.

Macro	Description
PH1D	Enables parallel solution of the 1D Helmholtz problems using the Schumann and Strietzel [1995] algorithm. If disabled, additional data transposing is performed and Gaussian elimination is performed locally.
PH1D_BUFFER	Buffers the global communications used by the Schumann and Strietzel [1995] algorithm.
INTERMEDIATE_TRANSPOSES	Transposes back to the original decomposition before and after the 1D Helmholtz solves. This greatly increases the amount of communication, but decreases the number of individual send / receives. Cannot be used with PH1D.
PETSC	Performs the potential vorticity inversion using PETSc.
SUPERLU_DIST	Performs the potential vorticity inversion using the SuperLU_DIST parallel direct solver (via PETSc).

6.2 qg3p_params.F90

The majority of model options are set in the qg3p_params.F90 Fortran source file.

The following control the model grid and partitioning.

Parameter	Description
`layers`	The number of model layers.
`iis`	An array of meridional domain sizes, defining the partitioning in the meridional direction.
`jjs`	An array of zonal domain sizes, defining the partitioning in the zonal direction.
`ii_m`	The maximum value of `iis`. Required for memory allocation.
`jj_m`	The maximum value of `jjs`. Required for memory allocation.
`periodic`	Whether the domain is periodic in the zonal direction.

The parameters `iis`, `jjs`, `ii_m` and `jj_m` can be generated using the `qg_decomp` script (see section 8.1).

The following control timestepping options.

Parameter	Description
`dt`	The model timestep size, in s. If using adaptive timestepping with CABARET, this is the size of the first timestep.
`adapt_dt`	Whether to use adaptive timestepping. Available only with CABARET.
`max_dt`	The maximum timestep size when using adaptive timestepping, in s. Available only with CABARET.
`max_dt_increase`	The maximum factor by which the timestep may increase in each timestep adapt. Available only with CABARET.
`target_cfl`	The target Courant number when using adaptive timestepping. Available only with CABARET.
`ra_nu`	The Robert-Asselin filter strength (μ in equation (3.8)). Available only with the centred scheme.
`ra_alpha`	The Robert-Asselin-Williams filter parameter (α in equation (3.8)). Available only with the centred scheme.
`set_start_time`	How to set the initial total model time. Has two possible values: `START_TIME_AUTO` and `START_TIME_MANUAL`. `START_TIME_AUTO` sets the initial total model time equal to zero or, if `start` is set to `START_PICKUP`, equal to the total model time in input checkpoint files. `START_TIME_MANUAL` allows one to override the initial total model time, and set it equal to a custom value.
`start_time_override`	The initial total model time, in days, if `set_start_time` is set to `START_TIME_MANUAL`.
`max_spin`	Model spin-up time, in days. During this time model time-average and eddy diagnostics are disabled.
`max_time`	Run-time of the model after spin-up, in days. The total model run-time is `max_spin` plus `max_time`.

The following control the physical parameters in the model. Note that wind

forcing in PEQUOD takes the form of:

$$F_{W,1} = -\frac{\pi\tau_0}{L_y^* h_1} A_W \sin\frac{\pi\left(L_y^* + y^*\right)}{L_y^* + y_m} \qquad \text{for } y^* \leq y_m, \tag{6.1}$$

$$= +\frac{\pi\tau_0}{L_y^* h_1}\frac{1}{A_W} \sin\left(\frac{\pi\left(y^* - y_m\right)}{L_y^* - y_m}\right) \qquad \text{for } y^* > y_m, \tag{6.2}$$

where:

$$y_m = 2C_W L_y^* + B_W x^* \frac{L_y^*}{L_x^*}, \tag{6.3}$$

and where $L_x^* = L_x/2$, $L_y^* = L_y/2$, $x^* = x - L_x^*$, and $y^* = y - L_y^*$. A_W can be used to generate an asymmetric wind stress, B_W can be used to tilt the wind stress with respect to the x-axis, and C_W can be used to offset the wind stress with respect to the domain centre.

Parameter	Description
`basinscale`	The meridional extent of the model domain in cm. Internal cells are square. Hence, combined with `iis` and `jjs`, this determines the horizontal dimensions of the model domain.
`beta`	Dimensional β parameter, in $\mathrm{cm}^{-1}\ \mathrm{s}^{-1}$.
`set_strat`	How to set the stratification. Has two possible values: `STRAT_RD` and `STRAT_INPUT`. `STRAT_RD` sets the stratification parameters using further options in `qg3p_params.F90`. `STRAT_INPUT` reads stratification parameters from a parameter file (see section 6.3.1).
`H`	An array of layer thicknesses, in order of uppermost layer to lowermost layer. Used only if `set_strat` is set to `STRAT_RD`.
`Rd_bcl1`	First baroclinic deformation radius in cm. Used only if `set_strat` is set to `STRAT_RD` and if there is more than one layer.
`Rd_bcl2`	Second baroclinic deformation radius in cm. Used only if `set_strat` is set to `STRAT_RD` and if there are more than two layers.
`tau`	Wind stress magnitude strength per unit density, in $\mathrm{cm}^2\ \mathrm{s}^{-2}$ (τ_0 in equation (6.1)).
`wind_shift`	Parameter controlling the shift of the wind stress zero line from the domain mid-latitude (C_W in equation (6.3)).
`wind_tilt`	Parameter controlling the tilt of the wind stress zero line relative to the zonal direction (B_W in equation (6.3)).
`wind_asym`	Parameter controlling the asymmetry of the wind stress to the North and South of the wind stress zero line (A_W in equation (6.1)).
`set_shear`	How to set the background vertical shear. Has two possible values: `SHEAR_U` and `SHEAR_INPUT`. `SHEAR_U` sets the shear using further options in `qg3p_params.F90`. `SHEAR_INPUT` reads shear parameters from a parameter file (see section 6.3.2).
`U`	Background zonal velocity, setting the background vertical shear, in $\mathrm{cm}\ \mathrm{s}^{-1}$. Used only if `set_shear` is set to `SHEAR_U`.
`u_background_h`	Background zonal velocity function, setting the background horizontal shear, in $\mathrm{cm}\ \mathrm{s}^{-1}$. The argument `y` varies from zero at the south edge of the domain to unity at the north edge of the domain.
`visc`	Laplacian viscosity coefficient, in $\mathrm{cm}^2\ \mathrm{s}^{-1}$.
`visc_bot`	Bottom friction coefficient, in s^{-1}.

The following control the model boundary conditions.

Parameter	Description
`partial_slip`	Whether to use partial slip boundary conditions. Otherwise no-slip boundary conditions are used.
`alpha`	Partial slip length scale, in cm ($1/\alpha$ in equation (2.28)). Unused if `partial_slip` is set to `.false.` .

Finally, the following control model initialisation and output.

Parameter	Description
`k_out`	Time, in days, between output of diagnostics.
`k_save`	Time, in days, between output of model output files.
`k_dump`	Number of outputs written before a new model output file is created.
`k_cp`	Number of outputs written before a full model checkpoint is written. Applies only when using CABARET.
`quad`	Whether to compute and output model eddy diagnostics.
`start`	How to start the model. Has three possible values: `START_ZERO`, `START_PERTURB`, and `START_PICKUP`. `START_ZERO` initialises the model with zero relative potential vorticity and stream function. `START_PERTURB` starts the model with a random interior relative potential vorticity field. `START_PICKUP` initialises the model from a checkpoint (see section 6.3.3).
`zeta_perturb`	Maximum magnitude of perturbations of the relative potential vorticity when `start` is set to `START_PERTURB`, in units of s^{-1}.
`save_log`	If enabled, writes standard output and standard error to log files.

6.3 Supplementary parameter files

In addition to `config.h` and `qg3p_params.F90`, a number of supplementary parameter files may be used to configure the model. The use of these parameter files is optional, and is dependent upon options defined in `qg3p_params.F90`.

6.3.1 Stratification parameters

If `set_strat` is set to `STRAT_INPUT` in `qg3p_params.F90` (see section 6.2) then stratification parameters are read from a parameter file located in `./input/stratification` relative to the program working directory. Note that such a parameter file must be specified if using more than three model layers.

The stratification parameter file is a plain text file defining the layer thicknesses, and the stratification parameters s_i^- and s_i^+ defined in equation (2.16). The first line of the file is a single integer defining the number of layers. All remaining lines contain up to three floating point numbers, defining the stratification parameters in order of uppermost to lowermost layer. For each line the floating point numbers define, in order: the layer thickness h_i, the stratification parameter s_i^-, and the stratification parameter s_i^+. Note that s_i^- is not defined for the first layer, and s_i^+ is not defined for the last layer. The h_i are in units of cm, and the s_i^- and s_i^+ are in units of cm $^{-2}$.

The following is an example of a three layer stratification parameter file:

```
3
0.25E+5  0.11861888862549392E-12
0.75E+5  0.39539629541831307E-13  0.74701918925497074E-13
0.30E+6  0.18675479731374269E-13
```

This defines three layers with upper layer thickness 250 m, middle layer thickness 750 m and lower layer thickness 3000 m. The stratification parameters defined correspond to a first baroclinic deformation radius of 40 km and a second baroclinic deformation radius of 23 km.

6.3.2 Shear parameters

If `set_shear` is set to `SHEAR_INPUT` in `qg3p_params.F90` (see section 6.2) then vertical shear parameters are read from a parameter file located in `./input/u` relative to the program working directory.

The shear parameter file is a plain text file defining the background zonal velocity. The first line of the file is a single integer defining the number of layers. All remaining lines contain one floating point number, defining the background zonal velocity in order of uppermost to lowermost layer. The velocities are defined in units of cm s^{-1}.

The following is an example of a three layer shear parameter file:

```
3
6.0
3.0
0.0
```

This defines a background zonal velocity of 6 cm s^{-1} in the upper layer, 3 cm s^{-1} in the middle layer, and zero in the lower layer.

6.3.3 Checkpoint files

If `start` is set to `START_PICKUP` in `qg3p_params.F90` (see section 6.2) then the model is initialised using checkpoint files located in `./input` relative to the program working directory. The checkpoint files are formatted as per normal model output, but with the output file number neglected. Note that, when using CABARET, supplementary data files are also required. The frequency of output of supplementary data files is controlled by the `k_cp` parameter in `qg3p_params.F90` (see section 6.2).

Checkpoint files can be copied into `./input` using the `qg_pickup` script (see section 8.2).

Chapter 7

Model output and diagnostics

This chapter details the PEQUOD model output data and online diagnostics. The formatting and types of model output data are described in section 7.1. Additional diagnostics output to standard output (or to log files if the `log_output` option is enabled – see section 6.2) are described in section 7.2.

7.1 Model output

This section describes PEQUOD model output, excluding data output to standard output (or model log files if the `log_output` option is enabled in `qg3p_params.F90` – see section 6.2). Model array indexing and non-dimensionalisation are detailed in sections 7.1.1 and 7.1.2 respectively. The output file format is detailed in section 7.1.3, and the data output by the model in this format is listed in section 7.1.4.

7.1.1 Array indexing

PEQUOD uses array style indexing for model variables. That is, the latitudinal index `i` refers to the `i` th row of nodes counting southward from the north edge of the domain, the longitudinal index `j` refers to the `j` th column of nodes counting eastward from the west edge of the domain, the latitudinal domain size `ii` refers to the total number of nodes in the latitudinal direction, and the longitudinal domain size `jj` refers to the total number of nodes in the longitudinal direction[1].

7.1.2 Non-dimensionalisation

Most model data is non-dimensionalised. For a model variable with dimensions L^aT^b, where L is a length scale and T a time scale, to convert to cgs units one multiplies by `l_scale`^{a} `t_scale`^{b}. `l_scale` and `t_scale` are output to meta-data files, as described in section 7.1.3.

[1] The only exception to this convention is the indexing of the partitioning in the meta-data files, `p_i`, `p_j`, `p_ii`, and `p_jj` described in section 7.1.3

7.1.3 Output file format

This section describes the file format used to store output model data. For a given data set each process outputs two files: a plain text file containing meta-data, and a binary file containing the output data itself.

The meta-data file has name:

```
base[.process]
```

where `base` is a filename prefix common to a given data set and `process` is the process number (indexed from zero). The process number is absent for serial simulations. The following is output at the start of the meta-data file, in plain text.

```
        n     p_ii     p_jj        p      p_i      p_j
     g_ii     g_jj   layers
     ii_0     jj_0       ii       jj
  l_scale  t_scale
```

where the variables have the following values:

Variable	Type	Value
`n`	integer	The total number of processes.
`p_ii`	integer	The decomposition size in the longitudinal direction (the size of `jjs` in `qg3p_params.F90` - see section 6.2).
`p_jj`	integer	The decomposition size in the latitudinal direction (the size of `iis` in `qg3p_params.F90` - see section 6.2).
`p`	integer	The process number, indexed from zero.
`p_i`	integer	The longitudinal position of the process in the domain partitioning, counting eastward from the west edge of the domain and indexed from one.
`p_j`	integer	The latitudinal position of the process in the domain partitioning, counting northward from the south edge of the domain and indexed from one.
`g_ii`	integer	The global domain size in the latitudinal direction.
`g_jj`	integer	The global domain size in the longitudinal direction.
`layers`	integer	The number of model layers.
`ii_0`	integer	The index of the upper row of the local process array in the global array, counting southward from the north edge of the domain and indexed from zero.
`jj_0`	integer	The index of the left column of the local process array in the global array, counting eastward from the west edge of the domain and indexed from zero.
`ii`	integer	The local domain size in the latitudinal direction.
`jj`	integer	The local domain size in the longitudinal direction.
`l_scale`	double precision	Space normalisation factor.
`t_scale`	double precision	Time normalisation factor.

The remainder of the meta-data file consists of a list of variable names and variable shapes. The possible sizes for each variable index are:

Variable size	Value
`ii`	The local domain size in the latitudinal direction.
`jj`	The local domain size in the longitudinal direction.
`layer`	The number of model layers.
`interface`	The number of model interfaces (the number of layers minus one).
`leapfrog`	The number of timesteps stored by the model when leapfrogging (always equal to two).
`flux_i_ii`	The local domain size in the latitudinal direction for an array staggered in the longitudinal direction.
`flux_i_jj`	The local domain size in the longitudinal direction for an array staggered in the longitudinal direction.
`flux_j_ii`	The local domain size in the latitudinal direction for an array staggered in the latitudinal direction.
`flux_j_jj`	The local domain size in the longitudinal direction for an array staggered in the latitudinal direction.

The following is an example of a meta-data file for time-averaged model

data:

```
      24       24        1        0        1        1
     513      513        3
       0        0      513       21
  0.75e6   0.75e6
model_time_day
total_time_day
psi1_av   ii  jj
psi2_av   ii  jj
psi3_av   ii  jj
```

This describes output for process 0 of a 24 process simulation decomposed only in the longitudinal direction, with two scalar variables `model_time_day` and `total_time_day`, and three variables each defined for a single model layer.

The binary data file has name:

```
base.d[.process]
```

where `base` is a filename prefix common to a given data set and `process` is the process number (indexed from zero). The process number is absent for serial simulations. Each variable listed in the meta-data file is output, in order, to the binary data file in the form:

```
[header][data][footer]
```

`header` consists of a default Fortran integer listing the number of bytes stored in `data`. `data` is a binary array of eight byte double precision values. `footer` is a further default Fortran integer, again equal to the number of bytes stored in `data`. This formatting is consistent with Fortran unformatted binary output on most platforms. Arrays are output using column-major ordering. Model variables are output with array style indexing, with the latitudinal index `i` counting southward from the north edge of the domain and the longitudinal index `j` counting eastward from the west edge of the domain (see section 7.1.1).

7.1.4 Output files

There are three types of data output in format described in the previous section: snapshot data, time-averaged data, and eddy diagnostics.

Snapshot data files

Snapshot data files have filename `base`:

```
qg[layers]_[ii]_s.[index]
```

where `layers` is the number of model layers, `ii` is the model domain size in the latitudinal direction, and `index` is an integer. `index` counts from zero, and is incremented after every `k_out` × `k_save` days, where `k_out` and `k_save` are defined in `qg3p_params.F90` (see section 6.2).

When using the centred scheme snapshot data files contain the following variables with the indicated shapes (as defined in meta-data files – see section 7.1.3):

```
model_time_day
total_time_day
psi1              ii   jj   leapfrog
...
psi[layers]       ii   jj   leapfrog
zeta1             ii   jj   leapfrog
...
zeta[layers]      ii   jj   leapfrog
```

When using the CABARET scheme snapshot data files contain the following variables with the indicated shapes:

```
model_time_day
total_time_day
psi1              ii   jj
...
psi[layers]       ii   jj
zeta1             ii   jj
...
zeta[layers]      ii   jj
```

`model_time_day` is the time, in days, executed by the current instance of the model. When picking up a simulation from checkpoint data using the `START_PICKUP` option in `qg3p_params.F90` (see section 6.2) this variable returns to a value of zero. `total_time_day` is the total simulation time, in days. When picking up a simulation from checkpoint data using the `START_PICKUP` option in `qg3p_params.F90` the value of this variable is preserved. `psi[N]` is the stream function and `zeta[N]` is the relative potential vorticity, each defined for layer N. The `psi[N]` and `zeta[N]` are output in non-dimensional units (see section 7.1.2).

When using the centred scheme the `psi[N]` and `zeta[N]` are output at two time levels: the current model time level n and the previous time level $n-1$. This is indicated by the `leapfrog` index size. Using Fortran array indexing, `psi(:, :, 1)` contains data from time level $n-1$ and `psi(:, :, 2)` contains data from time level n (the time indicated by `model_time_day` and `total_time_day`).

Time-averaged data files

Time-averaged data files have filename `base`:

```
qg[layers]_[ii]_s_av
```

where `layers` is the number of model layers and `ii` is the model domain size in the latitudinal direction.

Time-averaged data files contain the time-averages of fields. The time-averages are taken only after the time indicated by the `max_spin` parameter in `qg3p_params.F90` (see section 6.2). Time-averaged data files contain the following variables (as defined in meta-data files – see section 7.1.3):

```
model_time_day
```

```
total_time_day
psi1_av          ii   jj
...
psi[layers]_av   ii   jj
```

`model_time_day` and `total_time_day` are defined as for snapshot data files. `psi[N]_av` is the time-averaged stream function in the Nth model layer, in non-dimensional units (see section 7.1.2).

Eddy diagnostic data files

Eddy diagnostic data files have filename `base`:

```
qg[layers]_[ii]_quad_av
```

where `layers` is the number of model layers and `ii` is the model domain size in the latitudinal direction.

Eddy diagnostic data files contain the time-averages of products of fields. The time-averages are only taken only after the time indicated by the `max_spin` parameter in `qg3p_params.F90` (see section 6.2). Combined with data from the time-averaged data files, these data enable eddy covariances to be computed. Eddy diagnostic data files contain the following variables (as defined in meta-data files – see section 7.1.3):

```
model_time_day
total_time_day
uu_av            ii   jj   layer
vv_av            ii   jj   layer
uu_i_av          ii   jj   interface
vv_i_av          ii   jj   interface
uv_av            ii   jj   layer
ur_av            ii   jj   layer
vr_av            ii   jj   layer
rr_av            ii   jj   layer
pr_av            ii   jj   layer
uz_av            ii   jj   layer
vz_av            ii   jj   layer
zz_av            ii   jj   layer
zr_av            ii   jj   layer
pp_av            ii   jj   layer
pz_av            ii   jj   layer
sc_ub_av         ii   jj   interface
sc_vb_av         ii   jj   interface
sc_bb_av         ii   jj   layer
sc_bb_i_av       ii   jj   interface
```

`model_time_day` and `total_time_day` are defined as for snapshot data files. The values of the remaining variables are listed below.

Diagnostic	Value
`uu_av`	$\overline{u^2}$ on layers.
`vv_av`	$\overline{v^2}$ on layers.
`uu_i_av`	$\overline{u^2}$ on interfaces.
`vv_i_av`	$\overline{v^2}$ on interfaces.
`ur_av`	$\overline{u\omega}$ on layers.
`vr_av`	$\overline{v\omega}$ on layers.
`rr_av`	$\overline{\omega^2}$ on layers.
`pr_av`	$\overline{\psi\omega}$ on layers.
`uz_av`	$\overline{u\zeta}$ on layers.
`vz_av`	$\overline{v\zeta}$ on layers.
`zz_av`	$\overline{\zeta^2}$ on layers.
`pp_av`	$\overline{\psi^2}$ on layers.
`pz_av`	$\overline{\psi\zeta}$ on layers.
`sc_ub_av`	The layer model discretisation of $\frac{f_0}{N_0^2}\overline{ub}$, equal to $\frac{1}{2}h_i s_i^+ \overline{(u_i + u_{i+1})(\psi_i - \psi_{i+1})}$ for each interface i.
`sc_vb_av`	The layer model discretisation of $\frac{f_0}{N_0^2}\overline{vb}$, equal to $\frac{1}{2}h_i s_i^+ \overline{(v_i + v_{i+1})(\psi_i - \psi_{i+1})}$ for each interface i.
`sc_bb_av`	The layer model discretisation of $\frac{1}{2N_0^2}\overline{bb}$ on layers, equal to $\frac{1}{4}\left[s_i^- \overline{(\psi_{i-1} - \psi_i)^2} + s_i^+ \overline{(\psi_i - \psi_{i+1})^2}\right]$ for each layer i, where the s_i^- term is omitted for the uppermost layer and the s_i^+ term is omitted for the lowermost layer.
`sc_bb_i_av`	The layer model discretisation of $\frac{1}{2N_0^2}\overline{bb}$ on interfaces, equal to $\frac{h_i s_i^+}{h_i + h_{i+1}} \overline{(\psi_i - \psi_{i+1})^2}$ for each interface i.

The velocity components on layers are computed via centred differencing, and the velocity components on interfaces are computed via averaging of the values on adjacent layers.

7.2 Additional diagnostics

This section describes model diagnostics that are output to standard output, or to model log files if the `log_output` option is enabled. The frequency of diagnostic output is controlled by the `k_out` option (see section 6.2).

These diagnostics always output the current model time, in units of days and units of years (365 days). When running with the CABARET scheme the model timestep size is also output, in units of seconds and units of hours. Energy and performance diagnostics are then output. Energy diagnostics are described in section 7.2.1 and performance diagnostics are described in section 7.2.2.

7.2.1 Energy diagnostics

The energy diagnostics record the layer kinetic energies and the total model potential energy.

The discrete layer integrated kinetic energy density for layer k is given by:

$$K_k = \sum_{i=2}^{N_i} \sum_{j=2}^{N_j} \frac{h_k}{8} \Big[\left(\psi_{i,j-1,k} - \psi_{i-1,j-1,k} + \psi_{i,j,k} - \psi_{i-1,j,k}\right)^2 \tag{7.1}$$

$$+ \left(\psi_{i-1,j,k} - \psi_{i-1,j-1,k} + \psi_{i,j,k} - \psi_{i,j-1,k}\right)^2 \Big] . \tag{7.2}$$

The model outputs $K_k / \left(L_x L_y H\right)$ in non-dimensional units (see section 7.1.2). The discrete interface integrated potential energy density for interface k is given by:

$$P_k = \sum_{i=1}^{N_i} \sum_{j=1}^{N_j} \frac{a_{i,j} h_k s_k^+}{2} \left(\psi_{i,j,k} - \psi_{i,j,k+1}\right)^2 , \tag{7.3}$$

where $a_{i,j}$ equals one for interior nodes, one quarter for corner nodes, and one half for other boundary nodes. The model outputs $\sum_k P_k / \left(L_x L_y H\right)$ in non-dimensional units (see section 7.1.2).

7.2.2 Performance diagnostics

The performance diagnostics record the time taken by the model in executing various sections of the code. The diagnostics are measured by the `mpi_wtime` MPI subroutine, called by the first process and excluding all initialisation times. The times reported are in units of seconds, and are given together with a percentage of the total timestep loop runtime (rounding down).

The following diagnostics are recorded when running in serial or parallel:

Diagnostic	Value
Total	The total time spent executing the main timestep loop.
Integration	The time spent integrating the quasi-geostrophic potential vorticity equation using either the centred scheme or the CABARET scheme.
Helmholtz	The time spent performing the potential vorticity inversion.
Diagnostics	The time spent computing time-averaged and eddy diagnostics.
Other	All other model costs. This includes model output.

When running in parallel additional performance diagnostics recording the cost of communication are output. The exact parallel performance diagnostics are dependent upon options defined in `config.h` (see section 6.1).

Diagnostic	Value
Transpose	The time spent, by the first process, waiting for transpose communications to complete for the parallel Helmholtz solver.
1D Helmholtz comm	The time spent, by the first process, communicating data for the parallel 1D Helmholtz solver.
Halo wait	The time spent, by the first process, waiting for halo communication to complete.

Large values for the communication costs indicate an inefficient domain decomposition. See section 4 for further details regarding the parallelisation.

Chapter 8

Tools

A small number of Python scripts are supplied with PEQUOD. Note that to use these scripts a working installation of Python is required (see section 5.1.2).

8.1 qg_decomp

`qg_decomp` generates a PEQUOD domain decomposition for use in `qg3p_params.F90` (see section 6.2). `qg_decomp` is used via:

```
qg_decomp [OPTIONS] g_ii g_jj p_ii p_jj
```

where `g_ii` is the number of nodes in the longitudinal direction, `g_jj` is the number of nodes in the latitudinal direction, `p_ii` is the number of processes in the longitudinal direction, and `p_jj` is the number of processes in the latitudinal direction[1].

`qg_decomp` accepts the following options:

Option	Description
`-h`, `--help`	Display help and exit.

8.2 qg_pickup

`qg_pickup` copies model data from a previous simulation for use in restarting a new simulation. The data is copied to `input/` relative to the current working directory. In addition `qg_pickup` copies `config.h` and `qg3p_params.F90` to the current working directory, and modifies the latter so that the `start` option is set to `START_PICKUP` (see section 6.2). Note that `qg_pickup` does not modify the `set_start_time` option. `qg_pickup` is used via:

```
qg_pickup [OPTIONS] DIR
```

[1] Note that this differs from the definition of `p_ii` and `p_jj` in meta-data files – see section 7.1.3.

where `DIR` is the input directory.

`qg_pickup` accepts the following options:

Option	Description
`-h`, `--help`	Display help and exit.
`-d`, `--data`	Copy only input data files, and not configuration files.
`-n NPROC`, `--nproc=NPROC`	Specify number of processes in the input simulation.
`-s`, `--supplementary`	Require supplementary data files.

The `-n` option is required.

8.3 qg_merge_parallel

`qg_merge_parallel` combines parallel data files into a single serial data file. `qg_merge_parallel` is used via:

```
qg_merge_parallel [OPTIONS] BASENAME
```

where `BASENAME` is the model filename base (see section 7.1.4).

`qg_merge_parallel` accepts the following options:

Option	Description
`-h`, `--help`	Display help and exit.

`qg_merge_parallel` requires the Python NumPy module.

8.4 qg_cleanup

`qg_cleanup` deletes model data, except for the final model output. `qg_cleanup` is used via:

```
qg_pickup [OPTIONS] DIR
```

where `DIR` is the directory containing data for deletion.

`qg_cleanup` accepts the following options:

Option	Description
`-h`, `--help`	Display help and exit.
`-n NPROC`, `--nproc=NPROC`	Specify number of processes in the input simulation.
`-s`, `--supplementary`	Delete supplementary data files.

The `-n` option is required. When using the `-s` option `qg_pickup` deletes model data, including supplementary data files, except for the final model output with accompanying supplementary data files. Note that `qg_pickup` does not check that model times in the data files match.

WARNING: `qg_cleanup` may result in permanent data loss.

Bibliography

A. Arakawa. Computational design for long-term numerical integration of the equations of fluid motion: Two-dimensional incompressible flow. Part I. Journal of Computational Physics, 1(1):119–143, 1966.

R. Asselin. Frequency filter for time integrations. Mon, 100(6):487–490, 1972.

F. P. Bretherton. Critical layer instability in baroclinic flows. Quarterly Journal of the Royal Meteorological Society, 92(393):325–334, 1966.

G. J. Haltiner and R. T. Williams. Numerical Prediction and Dynamic Meteorology. John Wiley & Sons, 2 edition, 1980.

A. McC. Hogg, Blundell J. R., W. K. Dewar, and P. D. Killworth. Formulation and users' guide for Q-GCM Version 1.3.1, 2006.

S. A. Karabasov and V. M. Goloviznin. A new efficient high-resolution method for non-linear problems in fluid mechanics. In New Trends in Fluid Mechanics Research, pages 269–272. Springer, 2007.

S. A Karabasov, P. S. Berloff, and V. M. Goloviznin. CABARET in the ocean gyres. Ocean Modelling, 30(2-3):155–168, 2009.

J. C. McWilliams. A note on a consistent quasigeostrophic model in a multiply connected domain. Dynamics of Atmospheres and Oceans, 1(5):427–441, 1977.

J. Pedlosky. Geophysical fluid dynamics. Springer-Verlag, 2 edition, 1987.

A. J. Robert. The integration of a low order spectral form of the primitive meteorological equations. Journal of the Meteorological Society of Japan, 44 (5):237–245, 1966.

T. Schneider, I. M. Held, and S. T. Garner. Boundary effects in potential vorticity dynamics. Journal of the Atmospheric Sciences, 60(8):1024–1040, 2003.

U. Schumann and M. Strietzel. Parallel solution of tridiagonal systems for the Poisson equation. Journal of Scientific Computing, 10(2):181–190, 1995.

G. Strang. Linear algebra and its applications. Thomson Learning, Inc., 3 edition, 1988.

G. K. Vallis. Atmospheric and oceanic fluid dynamics: Fundamentals and large-scale circulation. Cambridge University Press, 2006.

P. D. Williams. A proposed modification to the Robert-Asselin time filter. Monthly Weather Review, 137(8):2538–2546, 2009.